

Learn JavaScript: Beginner's Course

Lesson 6: Arrays and Array Methods

Arrays and Array Methods - Study Notes

Key Concepts

- **Array creation** – literal syntax `[]` and `new Array()`; initializing with values.
- **Mutating methods** – `push()`, `pop()`, `shift()`, `unshift()`, `splice()` that change the original array.
- **Iterative (non mutating) methods** – `map()`, `filter()`, `forEach()`, `reduce()` that return new values or side effects without altering the source array.
- **Index based access** – retrieving/setting elements via `array[index]`; `length` property reflects number of elements.
- **Higher order functions** – passing callbacks to array methods; understanding parameters (`element`, `index`, `array`).

Summary

Arrays are the fundamental ordered collection type in JavaScript, allowing you to store multiple values under a single variable name. You can create an array with a literal (`[1, 2, 3]`) or the `Array` constructor, then manipulate its contents using built in methods. Mutating methods such as `push` and `pop` add or remove items from the end of the array, directly changing its length and contents.

When you need to transform or select data without altering the original array, you turn to higher order methods like `map` (creates a new array by applying a function to each element), `filter` (builds a new array with elements that pass a test), and `forEach` (executes a side effect for each item). Understanding the difference between mutating and non mutating methods is crucial for writing predictable code, especially when working with state in frameworks like React.

Finally, arrays expose useful properties such as `length` and support index based access (`arr[0]`). Mastering these basics prepares you for more advanced topics like multidimensional arrays, array like objects (e.g., `arguments`, `NodeList`), and functional programming patterns.

Important Points

- Use **array literals** (`[]`) for simplicity; they are faster and clearer than `new Array()`.
- `push()` adds to the **end**; `pop()` removes from the **end**.
- `unshift()` adds to the **beginning**; `shift()` removes from the **beginning** (both are $O(n)$ operations).
- `splice(start, deleteCount, ...items)` can insert, remove, or replace elements at any index.
- `map(callback)` returns a **new array** of the same length; the callback should return a value.
- `filter(callback)` returns a **new array** containing only elements where the callback returns `true`.
- `forEach(callback)` executes the callback for each element but returns `undefined`; use it for side effects (e.g., logging).
- Callback parameters: `(element, index, array)` – you can ignore any you don't need.
- Never rely on mutating methods when you need to preserve the original array; prefer non mutating alternatives or make a copy first (`arr.slice()` or `[...arr]`).

- The `length` property is **writable**; setting it to a smaller value truncates the array.

Examples

Example 1: Basic Mutating Operations

```
``javascript
let numbers = [1, 2, 3]; // ! [1, 2, 3]
numbers.push(4);         // add to end ! [1, 2, 3, 4]
numbers.pop();           // remove last ! [1, 2, 3]
numbers.unshift(0);      // add to start ! [0, 1, 2, 3]
numbers.shift();         // remove first ! [1, 2, 3]
// Insert 99 at index 1, delete 0 items
numbers.splice(1, 0, 99); // ! [1, 99, 2, 3]
````
```

**Explanation:**

- `push`/`pop` work on the tail; `unshift`/`shift` work on the head but shift all existing indices, making them slower for large arrays.
- `splice` is versatile: the first argument is the start index, the second is how many items to delete, and any additional arguments are items to insert.

### Example 2: Using map, filter, and forEach

```
``javascript
const words = ['apple', 'banana', 'cherry', 'date'];
// 1. map – get lengths
const lengths = words.map(w => w.length);
// lengths ! [5, 6, 6, 4]
// 2. filter – keep words longer than 4 letters
const longWords = words.filter(w => w.length > 4);
// longWords ! ['apple', 'banana', 'cherry']
// 3. forEach – log each word in uppercase
words.forEach(w => console.log(w.toUpperCase()));
// Output: APPLE, BANANA, CHERRY, DATE (each on its own line)
````
```

Explanation:

- `map` creates a new array (`lengths`) by applying the callback to each element.
- `filter` builds a new array (`longWords`) containing only those elements where the callback returns `true`.
- `forEach` does not create a new array; it's ideal for performing actions like logging, DOM updates, or accumulating state outside the array.

Quick Review

- What does `push` return?**
 - The new length of the array after the element is added.
- How would you create a copy of an array without mutating the original?**
 - Use the spread operator: `const copy = [...original];` or `slice()`: `const copy = original.slice();`

3. **Which array method would you use to produce a new array where each element is doubled?**

- `map`: `const doubled = arr.map(x => x * 2);`

4. **True or False:** `forEach` can be used to transform an array and return the transformed version.

- **False.** `forEach` returns `undefined`; use `map` for transformation.

5. **If you run `arr.splice(2, 3)`, what happens?**

- Starting at index 2, three elements are removed from `arr` and returned as a new array; the original array is shortened accordingly.

Further Reading

- **Array-like objects** – `arguments`, `NodeList`, and how to convert them to true arrays (`Array.from`).

- **Multidimensional arrays** – matrices, nesting, and common operations (e.g., flattening with `flat`).

- **Advanced iteration methods** – `reduce`, `reduceRight`, `some`, `every`, `find`, `findIndex`.

- **Immutability patterns** – using `concat`, `slice`, `spread`, and libraries like Immer for state management.

- **Performance considerations** – when to prefer `for` loops over array methods, and the cost of `unshift`/`shift` on large data sets.

End of study notes. Happy coding!

